

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study N-gram model	
Experiment No: 7	Date:	Enrolment No:92301733042

Aim: To study N-gram model

IDE: Google Colab

Theory:

Given a sequence of N-1 words, an N-gram model predicts the most probable word that might follow this sequence. It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input.

An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities. Since a simple N-gram model has limitations, improvements are often made via smoothing, interpolation and backoff. An N-gram model is one type of a Language Model (LM), which is about finding the probability distribution over word sequences.

Consider two sentences: "There was heavy rain" vs. "There was heavy flood". From experience, we know that the former sentence sounds better. An N-gram model will tell us that "heavy rain" occurs much more often than "heavy flood" in the training corpus. Thus, the first sentence is more probable and will be selected by the model.

A model that simply relies on how often a word occurs without looking at previous words is called unigram. If a model considers only the previous word to predict the current word, then it's called bigram. If two previous words are considered, then it's a trigram model.

An n-gram model for the above example would calculate the following probability:

$$P(\text{'There was heavy rain'}) = P(\text{'There', 'was', 'heavy', 'rain'}) = P(\text{'There'})P(\text{'was' | 'There'})P(\text{'heavy' | 'There was'})P(\text{'rain' | 'There was heavy'})$$

Since it's impractical to calculate these conditional probabilities, using Markov assumption, we approximate this to a bigram model: $P(\text{'There was heavy rain'}) \sim P(\text{'There'})P(\text{'was' | 'There'})P(\text{'heavy' | 'was'})P(\text{'rain' | 'heavy'})$ In speech recognition, input may be noisy and this can lead to wrong speech-to-text conversions.

N-gram models can correct this based on their knowledge of the probabilities. Likewise, N-gram models are used in machine translation to produce more natural sentences in the target language.

When correcting for spelling errors, sometimes dictionary lookups will not help. For example, in the phrase "in about fifteen minuets" the word 'minuets' is a valid dictionary word but it's incorrect in this context. N-gram

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study N-gram model	
Experiment No: 7	Date:	Enrolment No:92301733042

models can correct such errors.

N-gram models are usually at word level. It's also been used at character level to do stemming, that is, separate the root word from the suffix. By looking at N-gram statistics, we could also classify languages or differentiate

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study N-gram model	
Experiment No: 7	Date:	Enrolment No:92301733042

between US and UK spellings. For example, 'sz' is common in Czech; 'gb' and 'kp' are common in Igbo. In general, many NLP applications benefit from N-gram models including part-of-speech tagging, natural language generation, word similarity, sentiment extraction and predictive text input.

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Program (Code):

```
import nltk
from nltk.util import ngrams
from collections import defaultdict, Counter
import pandas
nltk.download('punkt')

corpus = """
I like machine learning
I like deep learning
I enjoy machine learning
I enjoy data science
Machine learning is interesting
Deep learning is powerful
"""

tokens=nltk.word_tokenize(corpus.lower())
#create the n-gram model
n=2
bigrams=list(ngrams(tokens,n))
bigrams

#creating the probability table
model=defaultdict(Counter)
for w1,w2 in bigrams:
model[w1][w2]+=1
model

def predict_nextword(word):
    if word in model:
        next_words=model[word]
        prediction=max(next_words,key=next_words.get)
        return prediction
    else:
        return "No prediction available"
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study N-gram model	
Experiment No: 7	Date:	Enrolment No:92301733042

```
input_word="enjoy"
print(predict_nextword(input_word.lower()))
```

Results:

```
[('i', 'like'),
 ('like', 'machine'),
 ('machine', 'learning'),
 ('learning', 'i'),
 ('i', 'like'),
 ('like', 'deep'),
 ('deep', 'learning'),
 ('learning', 'i'),
 ('i', 'enjoy'),
 ('enjoy', 'machine'),
 ('machine', 'learning'),
 ('learning', 'i'),
 ('i', 'enjoy'),
 ('enjoy', 'data'),
 ('data', 'science'),
 ('science', 'machine'),
 ('machine', 'learning'),
 ('learning', 'is'),
 ('is', 'interesting'),
 ('interesting', 'deep'),
 ('deep', 'learning'),
 ('learning', 'is'),
 ('is', 'powerful')]
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study N-gram model	
Experiment No: 7	Date:	Enrolment No:92301733042

```
defaultdict(collections.Counter,
            {'i': Counter({'like': 2, 'enjoy': 2}),
             'like': Counter({'machine': 1, 'deep': 1}),
             'machine': Counter({'learning': 3}),
             'learning': Counter({'i': 3, 'is': 2}),
             'deep': Counter({'learning': 2}),
             'enjoy': Counter({'machine': 1, 'data': 1}),
             'data': Counter({'science': 1}),
             'science': Counter({'machine': 1}),
             'is': Counter({'interesting': 1, 'powerful': 1}),
             'interesting': Counter({'deep': 1})})
```

```
input_word="enjoy"
print(predict_nextword(input_word.lower()))

machine
```

Observation:

The N-gram model is a probabilistic language model used to predict the next word based on previous words. It works by calculating the frequency of word sequences in a corpus and estimating their probabilities. The model follows the Markov assumption, which simplifies computation by considering only a limited number of previous words.

Different types of N-grams such as unigram, bigram, and trigram provide varying levels of context. While unigram ignores context, bigram and trigram improve prediction accuracy by considering previous words. However, N-gram models have limitations like data sparsity and require techniques such as smoothing to improve performance.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study N-gram model	
Experiment No: 7	Date:	Enrolment No:92301733042

Post Lab Exercise:

Apply the probability distribution concept to predict the next word by taking sample document and seed phrases. Attach the code and screenshot. Write your observation here.

CODE:

```
from collections import defaultdict
import nltk
nltk.download('punkt')
```

Sample text

```
text = "there was heavy rain in the city and the rain caused flood in many areas there was heavy rain again"
```

Tokenize

```
tokens = text.split()
```

Build bigram model

```
bigram = defaultdict(list)
```

```
for i in range(len(tokens)-1):
```

```
    bigram[tokens[i]].append(tokens[i+1])
```

Function to predict next word

```
def predict_next(word):
```

```
    if word in bigram:
```

```
        freq = {}
```

```
        for w in bigram[word]:
```

```
            freq[w] = freq.get(w, 0) + 1
```

```
        # Sort by highest probability
```

```
        predicted = max(freq, key=freq.get)
```

```
        return predicted, freq
```

```
    else:
```

```
        return None, {}
```

Seed phrase

```
seed = ["there", "was", "heavy"]
```

Predict next word based on last word

```
next_word, probabilities = predict_next(seed[-1])
```

```
print("Seed Phrase:", " ".join(seed))
```

```
print("Possible Next Words with Frequency:", probabilities)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study N-gram model	
Experiment No: 7	Date:	Enrolment No:92301733042

```
print("Predicted Next Word:", next_word)
```

OUTPUT:

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
Seed Phrase: there was heavy
Possible Next Words with Frequency: {'rain': 2}
Predicted Next Word: rain
```

Observation

The model correctly predicted "rain" as the next word for the phrase *"there was heavy"*. This is because "heavy rain" appears more frequently in the text. The prediction is based purely on probability distribution (frequency count). The model does not understand meaning but relies on pattern repetition. If more data is provided, predictions become more accurate.